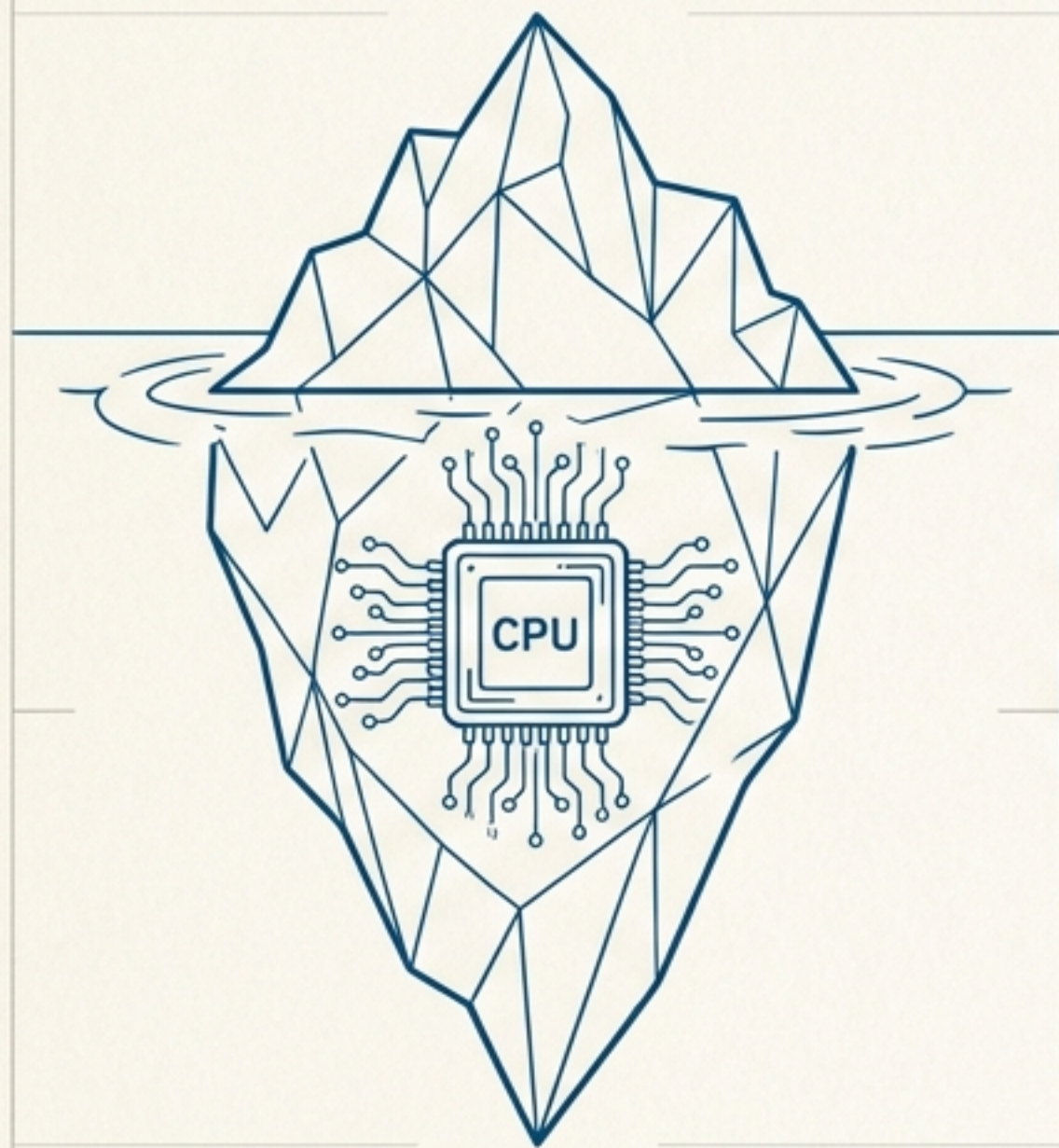




RAG 与长上下文： 大语言模型架构的未来

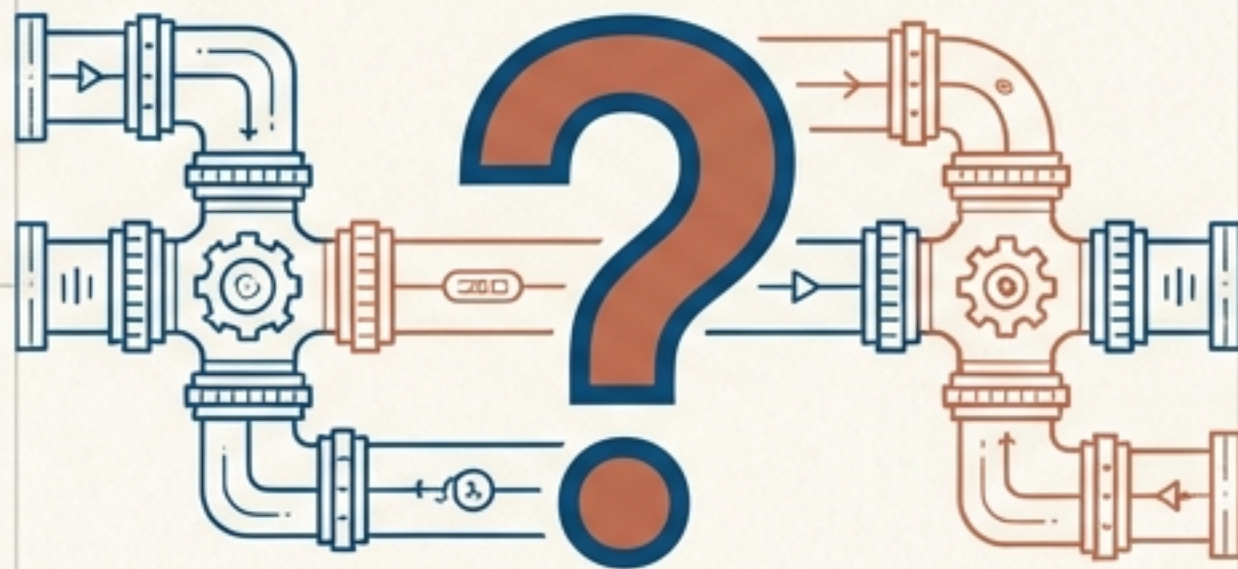
企业级 AI 的“语境注入”决策指南



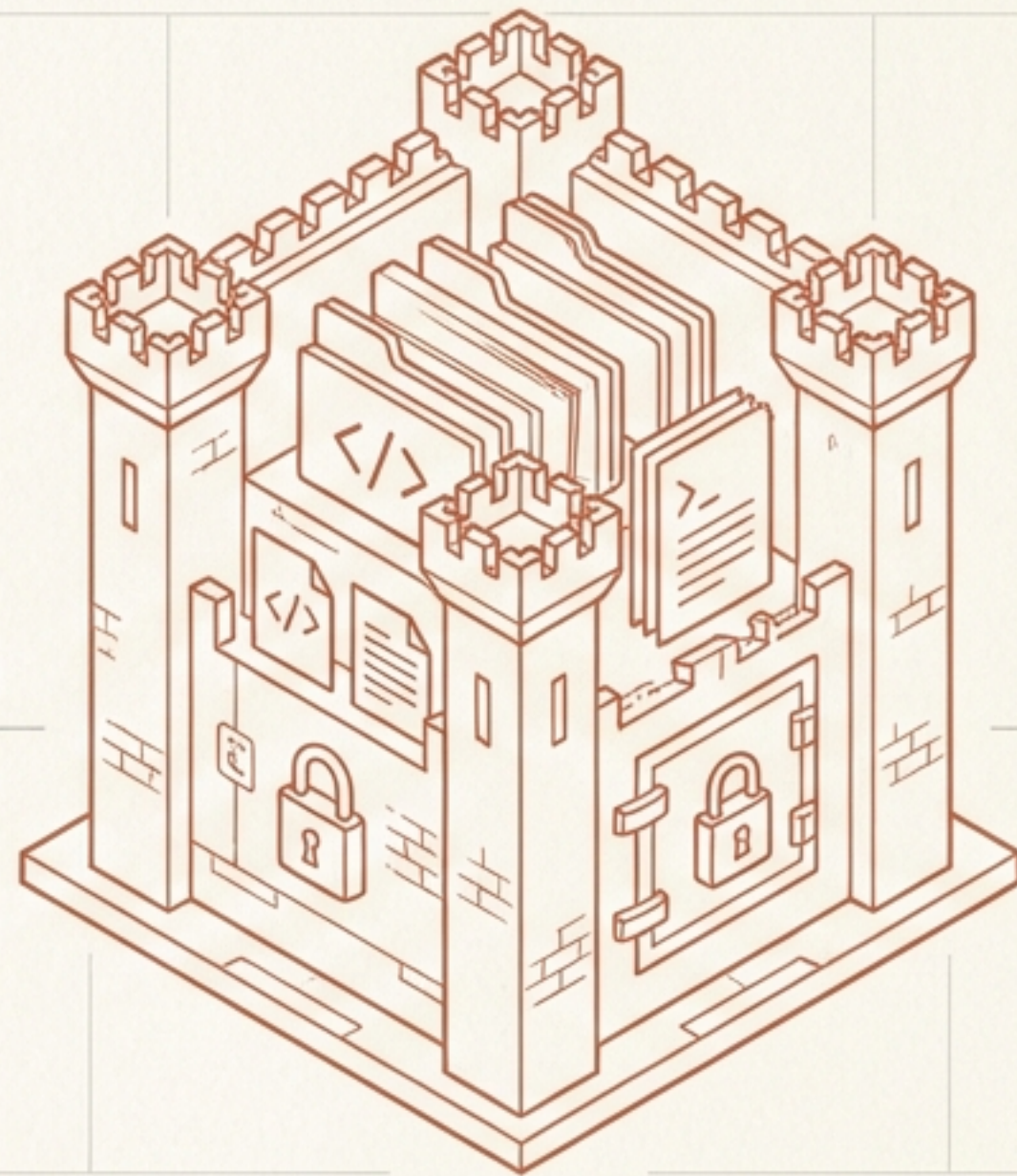
时间静止的 LLM

仅了解公开训练数据，对企业内部毫无所知

语境注入 (Context Injection)



如何在正确的时间将
正确的数据送达模型？

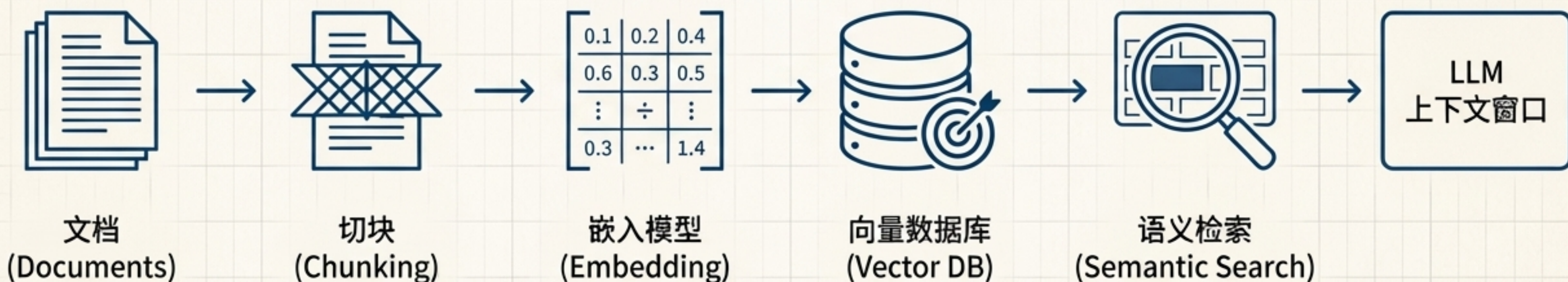


企业专属数据金库

私有文档、内部 Wiki、专有代码库

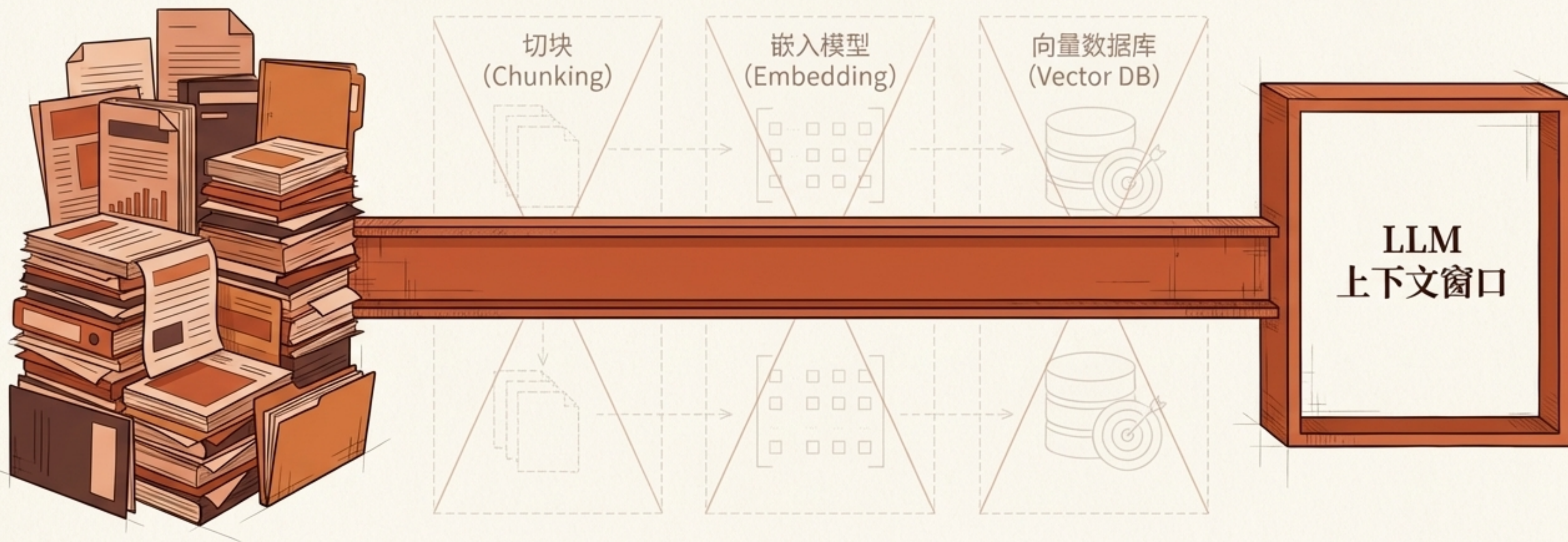
路径一：RAG（精准过滤机制）

工程化方案。将海量数据切割、转化为数学向量并存储。当用户提问时，系统通过检索逻辑提取最相关的碎片，与提示词一同注入大模型。它的成功极度依赖于检索逻辑能否找到正确信息。



路径二：长上下文（全景探照灯）

简单粗暴的模型原生方案。跳过数据库和嵌入模型，直接将所有文档放入上下文窗口。全权交由大模型的注意力机制（Attention Mechanism）来完成寻找答案的重任。



百万 Token 革命引发的架构之间

当我们可以简单地通过 Command C -> Command V 将所有文档直接粘贴给大模型时，RAG 是否已沦为不必要的复杂包袱？


4K Tokens
(早期模型)

1,000,000+ Tokens (现代模型)

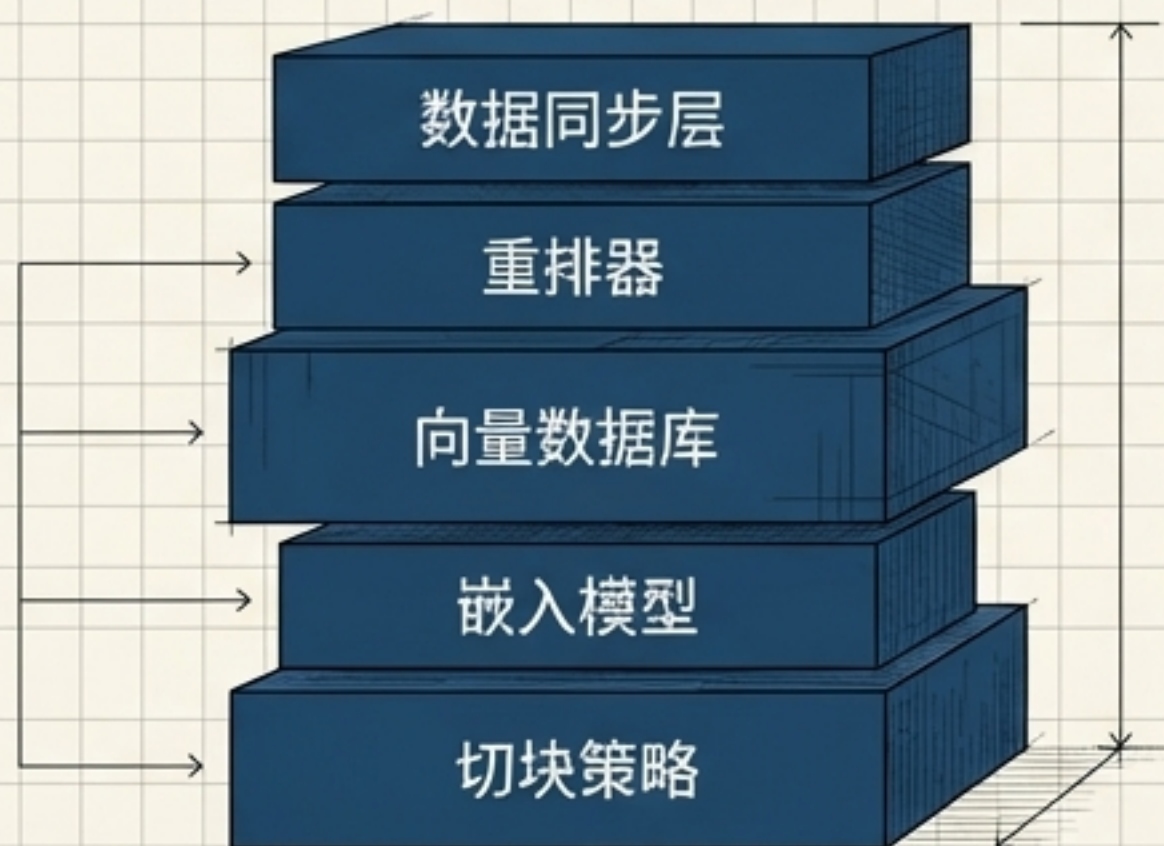


约 700,000 个单词 = 塞入整套《指环王》三部曲后，还有空间塞下《霍比特人》。

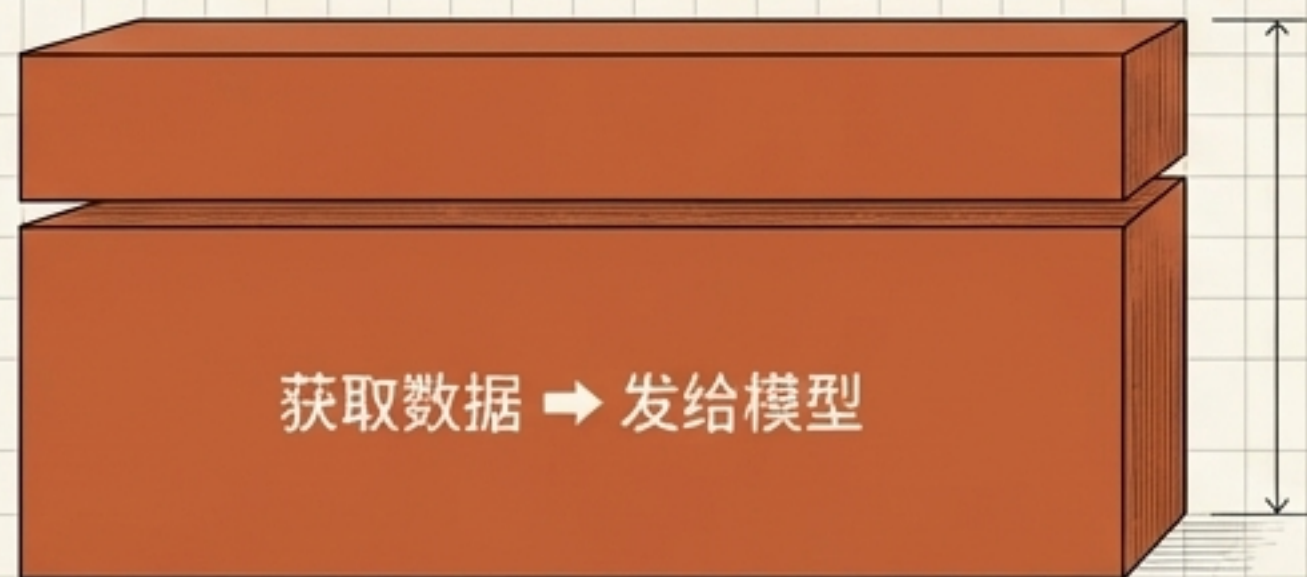


为什么选择长上下文？（一）基础设施的降维打击

生产级 RAG 系统包含大量移动部件。长上下文方案消除了数据库、嵌入和检索逻辑，将架构精简到极致。最纯粹的论点只有一个词：简单。



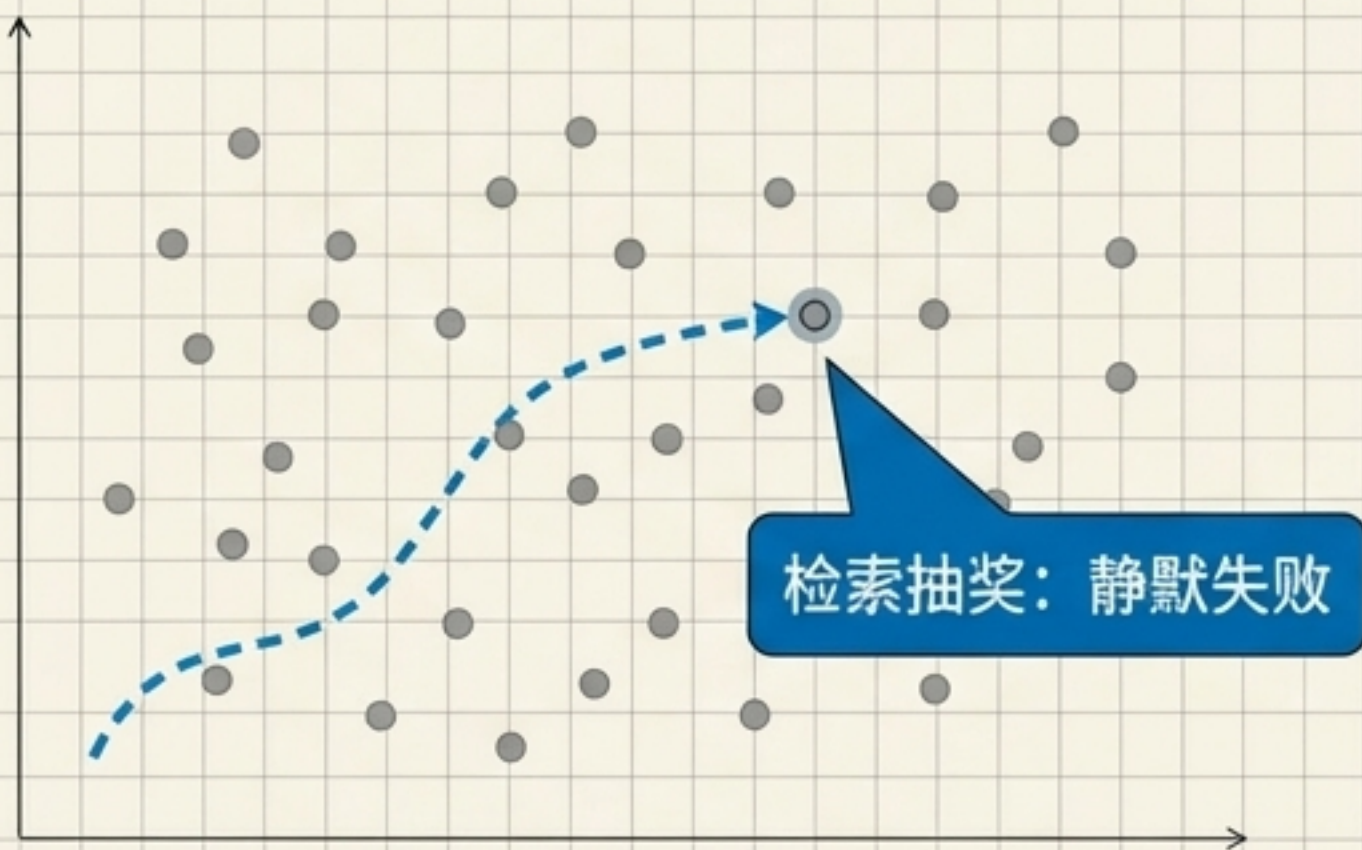
沉重且容易断裂的组件



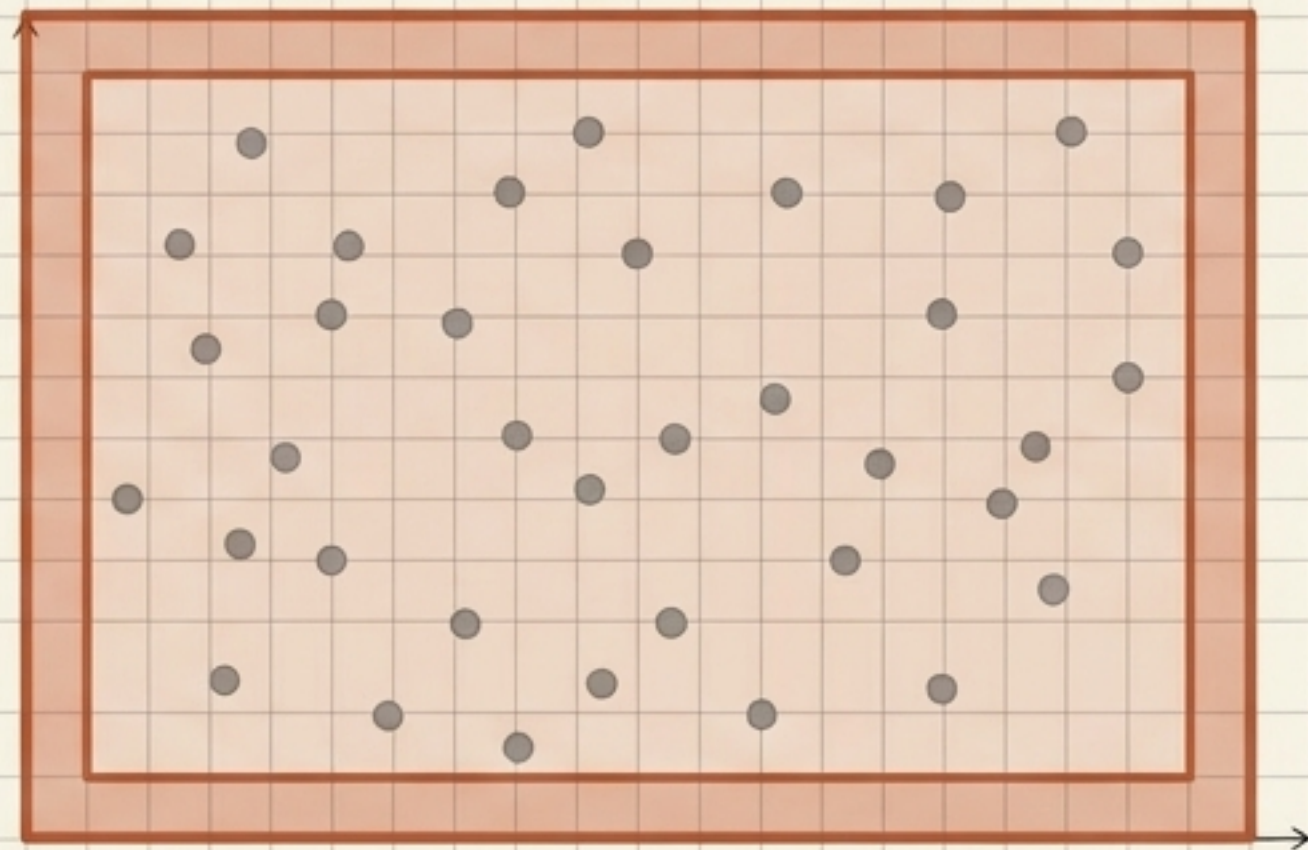
无栈之栈 (The No-Stack Stack)

为什么选择长上下文？（二） 终结“检索抽奖”

语义搜索是概率性的。RAG 经常遭遇静默失败（Silent Failure）：答案确实存在于数据中，但由于检索步骤匹配失败，大模型从未看到它。而在长上下文架构中，模型得以纵览全局。



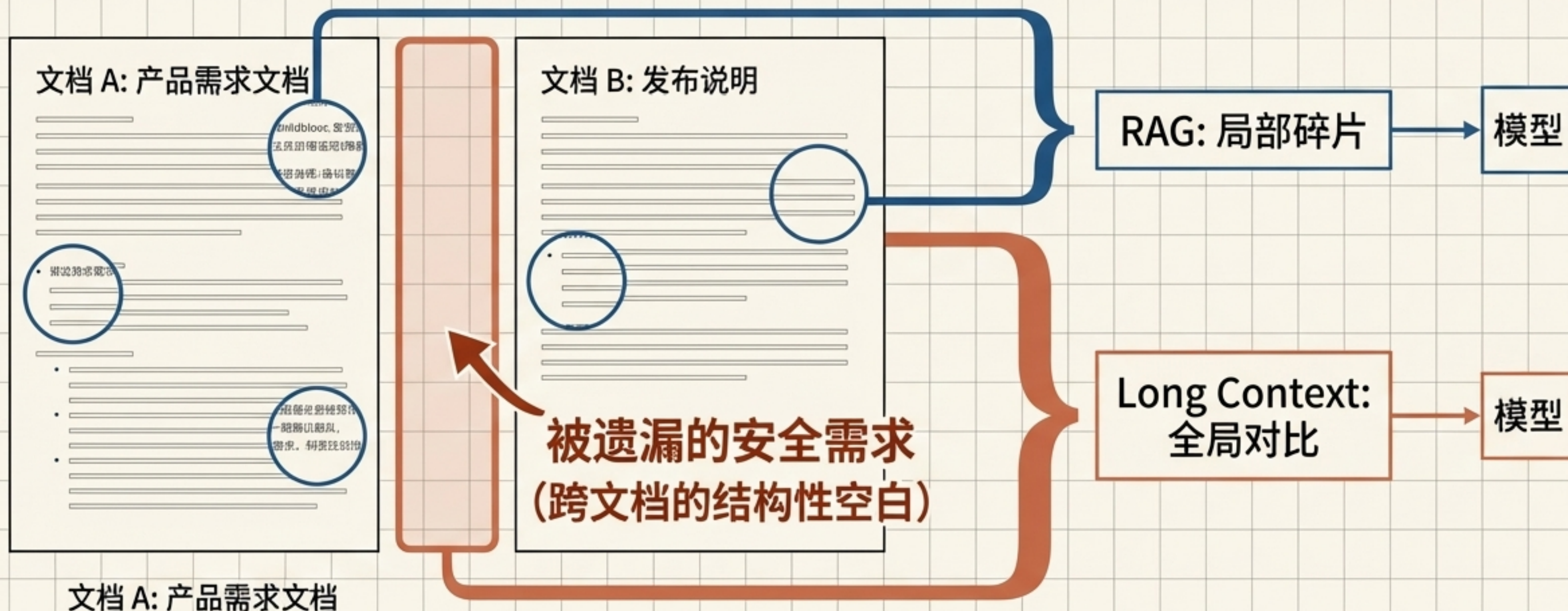
RAG

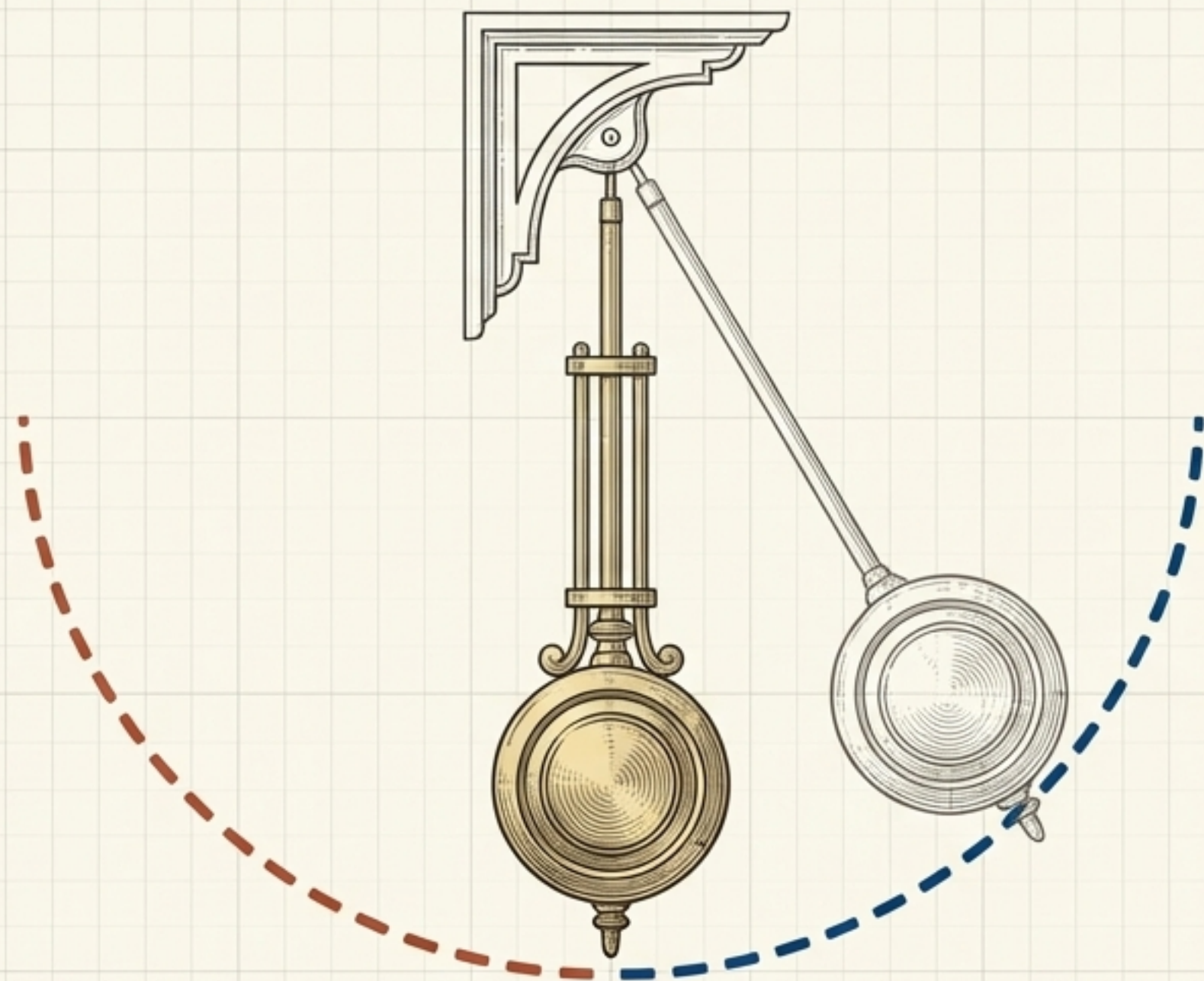


无检索步骤：纵览全局

为什么选择长上下文？（三）跨越“整书问题”的盲区

RAG 专为检索已存在的内容而设计。如果要求对比两份文档并找出遗漏项，RAG 仅能提供局部碎片，无法跨文档捕捉结构性空白。只有吞下整本书，模型才能进行全局对比。



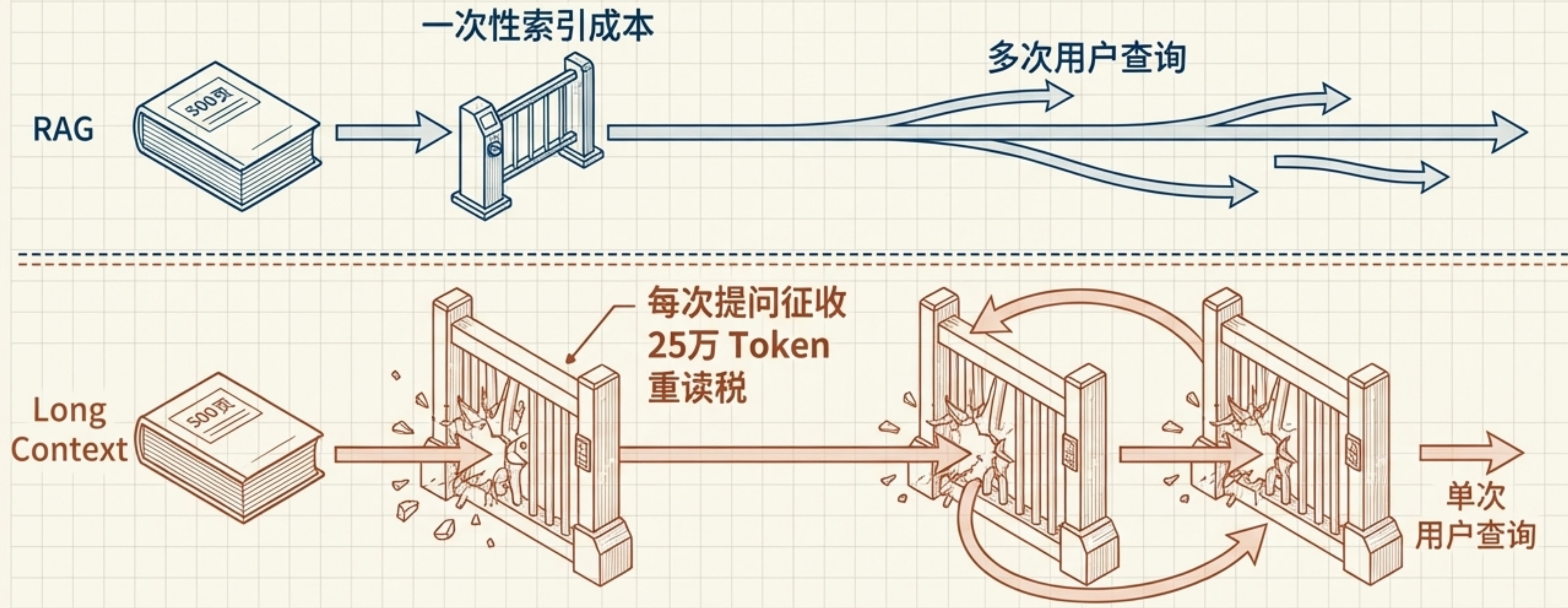


那么，向量数据库注定要进博物馆了吗？

尽管长上下文在“极简”上完胜，但 RAG 的核心工程逻辑依然坚不可摧。
以下是 RAG 将继续统治企业架构的三个致命理由。

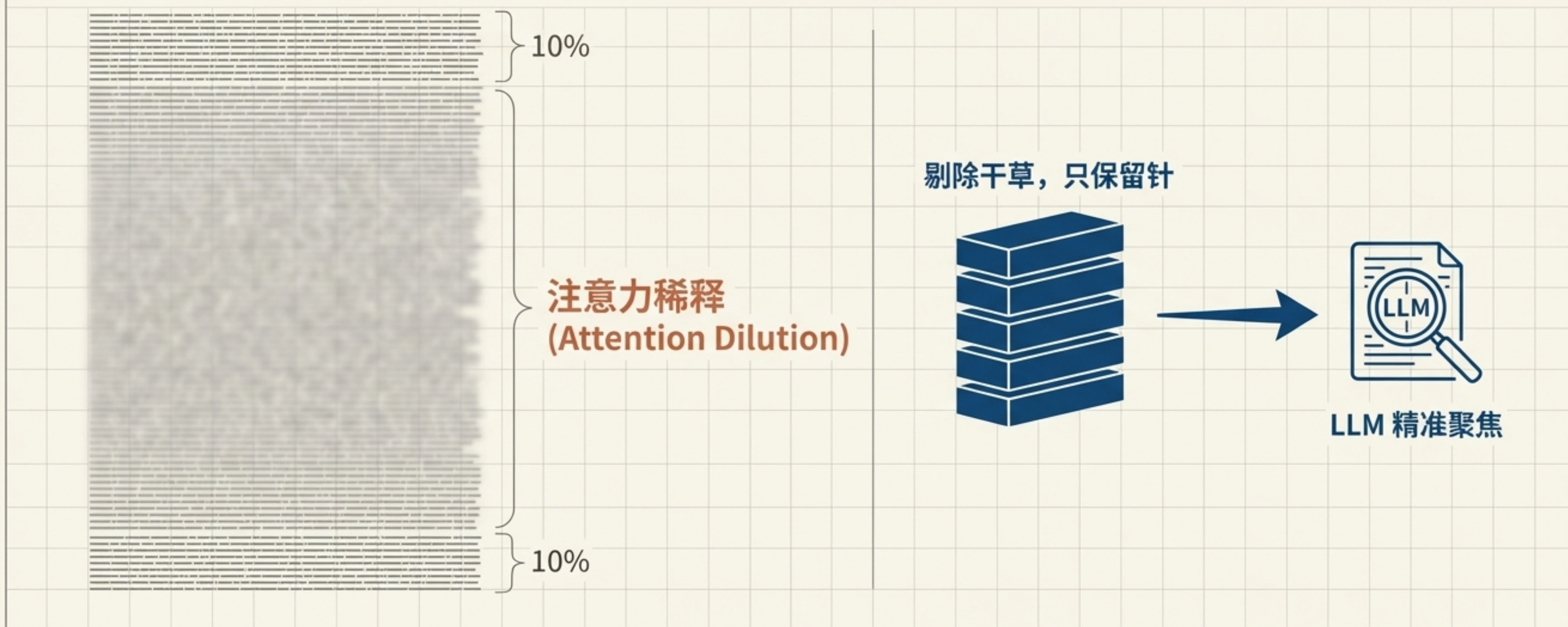
RAG 的绝地反击：（一）极其昂贵的“重读税”

每次提问时，长上下文都要求模型重新处理数十万 Token。对于动态数据流，这种巨大的算力低效等同于每次查询都要缴纳全额重读税。



RAG 的绝地反击：（二）破解“大海捞针”与注意力稀释

将所有数据扔进极宽的上下文窗口会稀释大模型的注意力机制。当关键信息被埋葬在数千页文档中间时，模型常常会遗忘。RAG 通过预先过滤噪音，强迫模型只关注核心信号。



RAG 的绝地反击：（三）驾驭 PB 级的无限数据集

百万 Token 的容量听起来惊人，但在以 PB 计量的企业知识库面前，仅仅是沧海一粟。面对真正的无限数据集，必须依靠向量数据库充当数据仓库与漏斗。

PB 级企业数据湖 (Petabyte Data Lake)



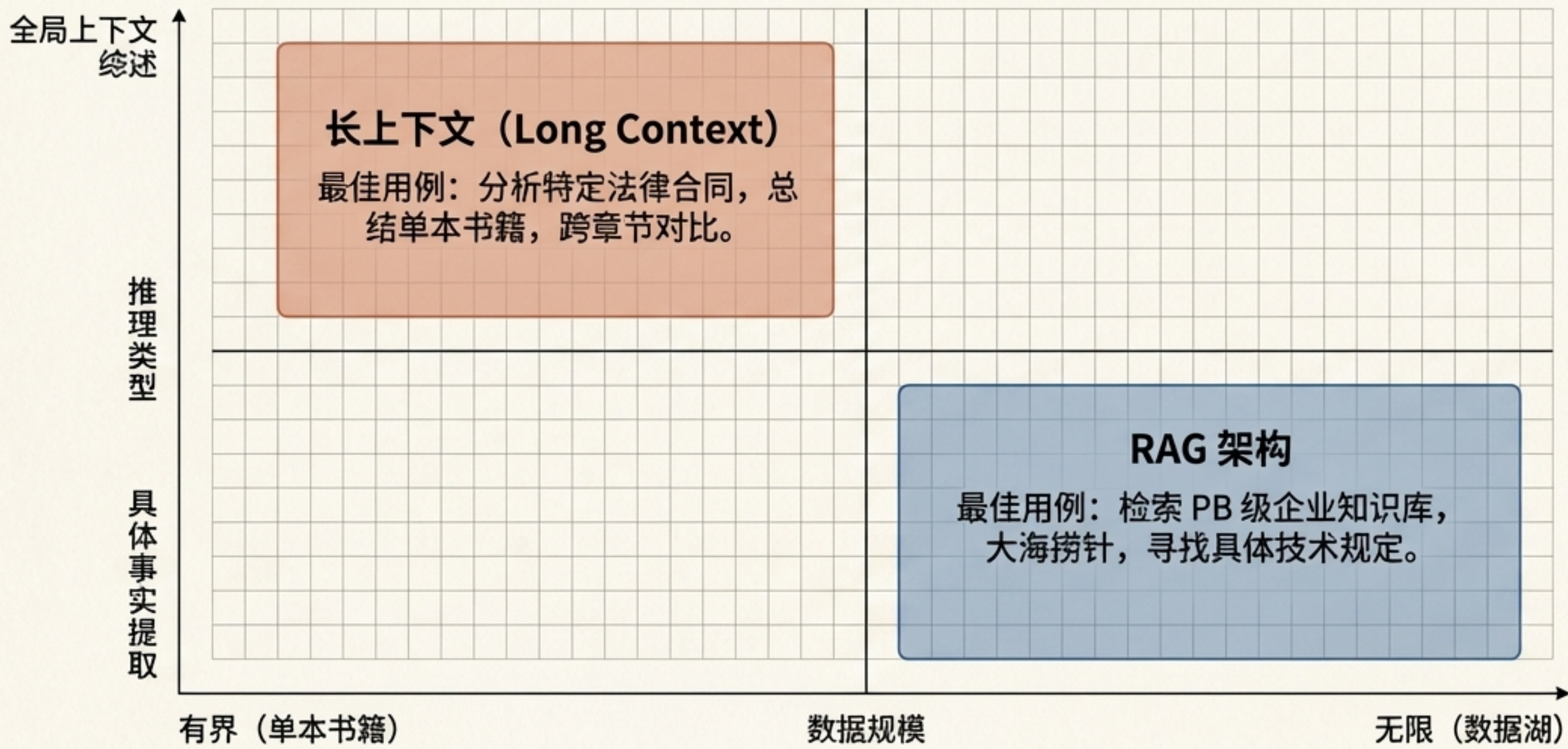
全局架构诊断矩阵

抛开“谁更好”的伪命题，通过工程维度的解构，评估两种架构的真实权衡。

	RAG 架构	长上下文
基础设施复杂度	极重（多移动部件）	极简（无栈架构）
算力成本模式	前置索引，单次查询成本低	极高的单次重读税
检索与准确性风险	存在静默失败风险	存在注意力稀释与幻觉风险
理想处理数据规模	无限量 / PB 级企业存储	有界限 / MB 级单次会话
推理与分析能力	局部 / 碎片化事实提取	全局 / 复杂跨文档逻辑

企业 AI 架构决策树

它们不是竞争对手，而是主宰不同业务象限的互补型工具。根据数据边界与推理深度，落子正确的架构。



终局：混合架构的觉醒

未来的企业 AI 不再是 RAG 与长上下文的非此即彼。最顶级的系统将融合两者：
用 RAG 作为无限数据湖的粗筛漏斗，用长上下文模型作为最终的精密全局分析引擎。
不要在工具之间站队，去设计融合两者的最优解。

